Week 1 · Day 2: Deep Learning

AI Intern Onboarding · Daily Task Sheet · Tuesday, 6 October 2026

Today's goal: understand how a neural network learns, and be able to explain in your own words what a neuron and a layer do, what happens in one pass of the training loop, how to spot and fix overfitting from a loss curve, and why embeddings and transfer learning matter. This feeds the Week 1 deliverable: embeddings and pretrained models are the ideas that Day 3 (transformers) and Day 4 (your Note Summariser) build on.

Data rule: use only public, de-identified or synthetic data today (and every day). No real patient information (PHI) in any exercise, notebook, prompt or daily note.

Schedule (shift 2:00 pm to 11:00 pm)

Time	Block	Activity
14:00–14:15	Setup	Open Jupyter, create a *week1-day2* notebook, check that torch and torchvision import, and note whether a GPU is available (torch.cuda.is_available()).
14:15–17:45	Study (3.5 h)	Topics 1–5 below, in order. Take short notes as you go; about 40 min per topic.
17:45–18:30	Meal break	
18:30–21:30	Hands-on (3 h)	Practice task: train a small network on MNIST and plot its loss curves (steps below).
21:30–22:30	Discussion (1 h)	Explain today's ideas to your peers and mentor (prompts below).
22:30–23:00	Daily note (0.5 h)	One page in your own words (template below).

Study topics

1. Neurons, layers and activation functions

- **What:** a neuron takes several numbers, multiplies each by a weight, adds them up with a bias, and passes the result through an activation function. A layer is many neurons side by side; a network stacks layers so each one builds on the features the last one found.
- **Why activations matter:** without a non-linear activation (ReLU, sigmoid, tanh), any stack of layers collapses into one linear model, no better than Day 1's logistic regression. ReLU is the usual default in hidden layers; sigmoid or softmax turns the final outputs into probabilities.
- **Where it breaks:** more layers mean more parameters, more data needed and less interpretability, which matters when a clinician asks why the model said what it said.
- **Do:** draw a network with 4 inputs, one hidden layer of 3 neurons and 1 output. Count its weights and biases by hand, then check with `sum(p.numel() for p in model.parameters())`.

2. The training loop: forward pass, loss, backpropagation, optimiser

- **What:** each step has four parts. **Forward pass:** run a batch through the network to get predictions. **Loss:** one number measuring how wrong they are (cross-entropy for classification, mean squared error for regression). **Backpropagation:** work out how much each weight contributed to the loss (its gradient). **Optimiser:** nudge every weight a little in the direction that lowers the loss (SGD, Adam).
- **Key words:** a batch is a small slice of the data; an epoch is one full pass over the training set; the learning rate is the size of each nudge.
- **Where it breaks:** too high a learning rate and the loss jumps around or explodes; too low and training crawls. Forgetting `optimizer.zero_grad()` in PyTorch silently adds up gradients across steps.

- **Do:** write the five lines of a PyTorch training step from memory (zero grads, forward, loss, backward, step) and label each with what it does.

3. Spotting and fixing overfitting: dropout, early stopping

- **Spotting it:** plot training loss and validation loss per epoch. Training loss keeps falling while validation loss flattens and then rises: the network is memorising instead of learning. Both stay high: it is underfitting.
- **Fixing it:** dropout randomly switches off some neurons during training so no single neuron can be relied on. Early stopping halts training when validation loss stops improving and keeps the best weights. Others to know by name: weight decay, more data, data augmentation, a smaller network.
- **Why it matters in healthcare:** an overfitted model can look excellent on the hospital it was trained on and fail at the next one.
- **Do:** sketch the two loss curves for an overfitting model and mark where early stopping would stop.

4. Embeddings: turning words or items into vectors

- **What:** an embedding is a list of numbers (a vector) that represents a word, sentence or item, learned so that similar things end up close together. "Fever" and "pyrexia" should sit near each other; "fever" and "invoice" should not.
- **Why it matters:** embeddings are how text becomes something a network can work with, and they power semantic search and RAG (Day 4).
- **Where it breaks:** embeddings learn the associations in their training text, including its biases and gaps; general-purpose embeddings may place medical abbreviations badly.
- **Do:** with `torch.nn.Embedding` or a small pretrained model, compute cosine similarity between a few word pairs and check whether the closest pairs make sense to you.

5. Transfer learning: starting from a pretrained model

- **What:** instead of training from random weights, start from a model already trained on a large dataset, then either freeze most of it and train only a new final layer (feature extraction) or retrain it gently at a small learning rate (fine-tuning).
- **Why it matters:** labelled medical data is scarce and expensive; transfer learning gets good results from far less of it. Every LLM you use this week is a pretrained model.
- **Where it breaks:** if the new task looks very different from the original data (everyday photos vs X-rays), less of the knowledge transfers; the pretrained model also brings its own biases.
- **Do:** write one sentence each on when you would freeze the pretrained layers and when you would fine-tune them.

Hands-on practice: train a small network on MNIST

Dataset: MNIST handwritten digits (70,000 public 28x28 greyscale images, no personal data), loaded with `torchvision.datasets.MNIST(download=True)`. It runs on a laptop CPU in a few minutes per epoch.

#	Step	Time
1	Load MNIST with torchvision; convert to tensors and normalise. Show 9 sample images with their labels and check the class balance.	20 min
2	Split the 60,000 training images into train (50,000) and validation (10,000) with a fixed seed. Keep the official 10,000 test images untouched until step 7. Wrap each in a DataLoader (batch size 64).	20 min
3	Define a small network with nn.Module: flatten, Linear(784, 128), ReLU, Linear(128, 64), ReLU, Linear(64, 10). Print it and count its parameters.	20 min
4	Write the training loop: cross-entropy loss, Adam optimiser (lr 0.001). Log training loss and validation loss after every epoch. Train for 15 epochs.	40 min

#	Step	Time
5	Plot training and validation loss on one chart. Mark the epoch where validation loss was lowest and write whether you see overfitting.	20 min
6	Add dropout (p=0.3) after each hidden layer and early stopping (patience 3, keep the best weights). Retrain and plot the new curves next to the old ones.	35 min
7	Evaluate the best model once on the test set: accuracy and a 10x10 confusion matrix. Which digits get confused with which?	15 min
8	Tidy the notebook so it reruns top to bottom with fixed seeds; save it.	10 min

Stretch (if time): swap the network for a small CNN (two Conv2d + MaxPool layers) and compare accuracy and parameter count. Or try learning rates of 0.1 and 0.00001 and plot what happens to the loss.

Challenge project: Transfer learning for medical image triage, with a model card

Difficulty: hard. Start it after the core practice task; expect roughly 8 to 10 extra hours, so work on it in spare slots across Week 1 and present it on Friday alongside your summariser. Ask your mentor early if you get stuck.

Scenario: a clinic wants a triage aid that flags chest X-rays or skin images for priority review. A missed abnormal image is far more harmful than an extra review. Compare training from scratch with transfer learning, evaluate honestly, and document the limits before anyone relies on it.

Data: public de-identified datasets only, e.g. MedMNIST (PneumoniaMNIST or DermaMNIST, installed with `pip install medmnist`). Use its official train/validation/test split. Never use images from a real clinic.

Milestones

#	Milestone	What to deliver
1	Data and baseline	Load the dataset, show samples per class, report class balance. Train a small CNN from scratch as your baseline. Explain in one paragraph why accuracy alone would mislead here.
2	Reproducible training loop	One train() function with fixed seeds, logged train/validation loss and recall per epoch, early stopping on validation loss, and checkpointing of the best weights.
3	Transfer learning	Start from a pretrained ResNet-18 (torchvision). Adapt the input and the final layer. Compare (a) frozen backbone, new head only and (b) full fine-tuning at a small learning rate. Report all three models side by side.
4	Regularisation study	Try data augmentation (small rotations, flips only where clinically sensible), dropout and weight decay. Show their effect on the gap between train and validation loss with plots.
5	Cost-sensitive threshold	Define a cost where a missed positive costs 10x a false alarm. Pick the threshold on validation data that minimises expected cost. Plot the precision-recall curve and mark your threshold.
6	Calibration and uncertainty	Plot a reliability curve and apply temperature scaling. Compare Brier score before and after. Run Monte Carlo dropout (dropout on at inference, 20 passes) and check whether high-uncertainty images are more often wrong.
7	Error analysis and explainability	Inspect every false negative on the test set. Use Grad-CAM (or a saliency map) on 10 correct and 10 wrong cases. Does the model look at the right region, or at text markers, borders or artefacts?
8	Model card	A 1-2 page markdown document: intended use, out-of-scope use, data source and resolution, metrics with uncertainty, threshold rationale, known failure modes, and the need for clinician review.

Success criteria

- Notebook or repo reruns end to end from a fresh environment (requirements file included, seeds fixed).
- Test set touched only once, for the final numbers.
- Every claim about performance has a number and an uncertainty estimate.
- Your model card states plainly where the model should not be trusted (e.g. image resolution, scanner type, populations not in the data).
- You can defend each design choice in a 10-minute walkthrough.

Stretch goals

- Compare against a trivial baseline (always predict the majority class) and a classic model on flattened pixels (logistic regression from Day 1).
- Compute bootstrap confidence intervals for recall on the test set.
- Extract embeddings from the penultimate layer, project them to 2D with t-SNE or UMAP, and colour by class. Do the classes separate?
- Test robustness: add blur, noise or brightness shifts to the test images and measure the drop in recall.

Resources to study from

Core (free)

Resource	Use it for
PyTorch tutorials: Learn the Basics (pytorch.org/tutorials)	Official walkthrough of tensors, datasets, DataLoaders, building a model, autograd, the optimisation loop and saving models. Closest match to today's hands-on.
3Blue1Brown: Neural Networks series (YouTube)	Visual intuition for neurons, layers, gradient descent and backpropagation, using MNIST as the example.
Google Machine Learning Crash Course (developers.google.com/machine-learning/crash-course)	Short lessons on neural networks, embeddings and overfitting.
StatQuest with Josh Starmer (YouTube)	Neural networks, backpropagation, ReLU and word embeddings, step by step.
Dive into Deep Learning (d2l.ai)	Free interactive book with PyTorch code: multilayer perceptrons, dropout, weight decay, fine-tuning.
fast.ai: Practical Deep Learning for Coders (course.fast.ai)	A top-down view of transfer learning in practice.

Deeper reading

- *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow* by Aurelien Geron: chapters on artificial neural networks and training deep networks.
- *Deep Learning* by Goodfellow, Bengio and Courville (free online at deeplearningbook.org): chapter on regularisation, for depth on dropout and early stopping.
- Andrew Ng's Deep Learning Specialization (Coursera): course 2 on improving deep networks (regularisation, optimisation, tuning).
- Srivastava et al., 'Dropout: A Simple Way to Prevent Neural Networks from Overfitting' (2014): the original dropout paper.
- Mikolov et al., 'Efficient Estimation of Word Representations in Vector Space' (2013): the word2vec paper behind modern embeddings.

- Selvaraju et al., 'Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization' (2017): the method in challenge milestone 7.
- Guo et al., 'On Calibration of Modern Neural Networks' (2017): why deep networks are overconfident, and temperature scaling.

Public practice datasets

- MNIST and Fashion-MNIST (torchvision built-ins) for the core task.
- MedMNIST (medmnist.com): small, standardised, de-identified medical image sets for the challenge.
- Check each dataset's licence and confirm it is de-identified before use.

Resource availability and content change over time; if a link or course has moved, search for the title or ask your mentor.

Check yourself

How can you tell from a loss curve that a model is overfitting?

Hint: look at where the training and validation curves go after the first few epochs. What does it mean when one keeps falling and the other turns upward? Write your answer, then point to the exact epoch on your own MNIST plot from step 5 where it starts happening (or explain why it does not).

Discussion prompts (1 hour)

- Explain backpropagation to a non-technical colleague in two minutes, using an analogy (e.g. a team working out who caused a mistake and how much).
- Your training loss is 0.01 and your validation loss is 0.40. What do you try first, and why?
- A model pretrained on everyday photos is fine-tuned on chest X-rays. What knowledge transfers well, and what might mislead it?
- Where could embeddings trained on general web text place medical terms or abbreviations badly? What could that cause in our Note Summariser?
- What surprised you when you added dropout and early stopping?

Daily note template (one page)

- Three things I learned today (my own words).
- One thing that is still unclear, to ask my mentor.
- My answer to the check-yourself question.
- What I will do differently tomorrow.

Definition of done today: notebook runs end to end; training and validation loss curves plotted before and after dropout and early stopping; test accuracy and confusion matrix for the final model; written explanation of any overfitting you saw; daily note submitted; questions logged for your mentor.

Looking ahead: Day 3 covers transformers and LLMs (tokens, attention, BERT vs GPT, how chat models are made, temperature and hallucination) with Hugging Face Transformers. Install transformers before your next shift if you can.